homework_1

October 14, 2026

1 Homework 1

DATA 37711 — written answers to Problems 1–3.

1.1 Problem 1

Setup. In the 2020 US Presidential election, poll aggregators (e.g., fivethirtyeight.com) and betting markets disagreed about the probability of a Biden victory. Under Bayesian decision theory, a rational agent writes down a probability distribution $P(X)$ over states of the world and a utility function $U(a, X)$, then chooses

$$a^* = \arg\max_a \ \mathbb{E}_{X\sim P}\big[U(a, X)\big].$$

Victor is a rational agent with d dollars of total wealth. He believes $P(\text{Biden wins}) = 0.85$ (the fivethirtyeight prediction), and betting markets give even-money bets at $P(\text{Biden wins}) = 0.6$.

1.1.1 (a)

Question. Prove that if Victor's utility function is just the amount of money he has, he'll bet d (his entire bankroll) on Biden winning. Does linear utility of money make sense?

Answer.

Payout. The market odds are fair for someone who believes $P(\text{Biden wins}) = 0.6$, so a bet of a dollars has zero expected profit under that belief:

$$0.6 \cdot \text{profit} - 0.4\, a = 0 \quad \Longrightarrow \quad \text{profit} = \tfrac{2}{3}\, a.$$

So each dollar bet on Biden earns 2/3 of a dollar if Biden wins.

Utility. Let $a \in [0, d]$ be the amount bet on Biden. Victor's final wealth is

$$U(a, \text{Biden wins}) = d + \tfrac{2}{3}a \quad \text{(he gets his stake back plus the profit)},$$
$$U(a, \text{Biden loses}) = d - a.$$

Expected utility under Victor's beliefs:

$$\mathbb{E}[U(a, X)] = 0.85\,\big(d + \tfrac{2}{3}a\big) + 0.15\,(d - a) = d + \big(0.85 \cdot \tfrac{2}{3} - 0.15\big)\, a = d + \tfrac{5}{12}\, a.$$

The feasible actions are $0 \le a \le d$, since he can't bet more than he has. The coefficient $5/12$ is strictly positive, so $\mathbb{E}[U]$ is strictly increasing in a, and its maximum on $[0, d]$ is at the endpoint $a^* = d$. Victor bets his entire bankroll. ■

(The coefficient is positive because Victor's belief, 0.85, exceeds the market's, 0.6; if they were equal, it would be zero and he would be indifferent.)

Does linear utility make sense? No. Under this strategy, Victor has a 15% chance of losing everything. People generally care more about avoiding ruin than about gaining an equal amount of money (risk aversion), so betting everything on one event is not reasonable. Also, the value of an extra dollar depends on how much you already have: a fixed gain means less to someone with large d than to someone with small d (diminishing marginal utility). Both ideas suggest utility should be a concave, nonlinear function of total wealth. Under such a utility, a reasonable agent with an edge would still bet something, but not everything.

1.1.2 (b)

Question. Now suppose Victor's utility from y dollars is $U(y) = \log(y)$. How much (of his d dollars) should he bet on Biden winning?

Answer.

Final wealth in each outcome is the same as in (a), so

$$U(a, \text{Biden wins}) = \log\!\left(d + \tfrac{2}{3}a\right), \qquad U(a, \text{Biden loses}) = \log(d - a).$$

Expected utility:

$$E(a) = 0.85 \log\!\left(d + \tfrac{2}{3}a\right) + 0.15 \log(d - a), \qquad 0 \le a < d.$$

First-order condition:

$$\frac{dE}{da} = \frac{0.85 \cdot \frac{2}{3}}{d + \frac{2}{3}a} - \frac{0.15}{d - a} = 0 \implies 0.85 \cdot \tfrac{2}{3}\,(d - a) = 0.15\left(d + \tfrac{2}{3}a\right).$$

Multiplying by 3:

$$1.7\,(d - a) = 0.45\,d + 0.3\,a \implies 1.25\,d = 2a \implies a^* = 0.625\,d = \tfrac{5}{8}\,d.$$

This is a maximum: log is concave, so each term of E is concave in a, which makes E concave and the critical point its global maximum on $[0, d)$.

So Victor bets 5/8 of his wealth. This is between 0 and d and less than the "bet everything" answer from (a): log utility makes him keep some money in reserve, but his edge still leads to a large bet. This matches the Kelly criterion, $f^* = p - (1 - p)/b$ with $p = 0.85$ and $b = 2/3$:

$$0.85 - \frac{0.15}{2/3} = 0.625.$$

[1]:
```
import numpy as np

# Numerical check of 1(b) with d = 1: expected log utility on a fine grid of a␣
↪in [0, d)
d = 1.0
a = np.linspace(0, d, 1_000_001, endpoint=False)  # a in [0, d)
expected_log_utility = 0.85 * np.log(d + (2 / 3) * a) + 0.15 * np.log(d - a)

a_star = a[np.argmax(expected_log_utility)]
print(f"grid maximizer a* = {a_star:.6f}  (analytic: 5/8 = 0.625)")
assert abs(a_star - 0.625) < 1e-4
```

```
grid maximizer a* = 0.625000  (analytic: 5/8 = 0.625)
```

1.1.3 (c)

Question. Suppose you had a net worth of $500,000. Would you personally make the bet from part (b)?

Answer (my opinion).

No, not at that size. 5/8 of $500,000 is $312,500, which means a 15% chance of losing more than $300,000 on a single event. Kelly is optimal only if the model's assumptions hold, and I don't think they fully apply to me:

- The 0.85 is an estimate, not a known fact. The market's 0.6 is also information, and if my probability is too high, full Kelly overbets, which is penalized much more than underbetting. That's why practitioners often use fractional Kelly (e.g., half Kelly).
- My personal utility is likely more risk-averse (more concave) than log, and under a more concave utility the optimal bet really is smaller.
- The model assumes this bet is my only option. In reality I'd rather keep most of my money in more reliable, diversified investments.

I would still bet on Biden if I held that conviction, but only a small percentage of my wealth.

1.2 Problem 2

Question. Suppose Y is a categorical random variable and X are covariates used to predict Y. A loss function L on categorical data is a *proper scoring rule* if the predictive function that minimizes the implied risk is the conditional probability of Y given X; that is, if

$$f^* := \arg\min_f \mathbb{E}_{X,Y}[L(Y, f(X))] \quad \text{is given by} \quad f^*(x) = P(Y \mid X = x).$$

Prove that the cross-entropy loss is a proper scoring rule when Y is binary.

Answer.

For binary $Y \in \{0, 1\}$, the prediction $f(x) = q \in (0, 1)$ is the predicted probability that $Y = 1$. The binary cross-entropy loss is

$$L(y, q) = -\big[\, y \log q + (1 - y) \log(1 - q) \,\big],$$

which is $-\log q$ when $y = 1$ and $-\log(1 - q)$ when $y = 0$.

Step 1 (law of total expectation).

$$\mathbb{E}_{X,Y}\big[L(Y, f(X))\big] = \mathbb{E}_X\Big[\, \mathbb{E}_{Y|X}\big[L(Y, f(X)) \mid X\big]\Big].$$

Step 2 (inner expectation). Fix x, and let $p = P(Y = 1 \mid X = x)$ and $q = f(x)$. Then

$$\mathbb{E}_{Y|X}\big[L(Y, q) \mid X = x\big] = -\big[\, p \log q + (1 - p) \log(1 - q) \,\big] =: g(q).$$

Step 3 (minimize over q).

$$g'(q) = -\frac{p}{q} + \frac{1 - p}{1 - q} = 0 \implies (1 - p)\, q = p\, (1 - q) \implies q = p.$$

Moreover,

$$g''(q) = \frac{p}{q^2} + \frac{1 - p}{(1 - q)^2} > 0 \quad \text{on } (0, 1),$$

so g is strictly convex and $q = p$ is its unique minimizer.

Step 4 (pointwise minimization suffices). Because f is an arbitrary function, its value at each x can be chosen independently of its value at any other x. For every x, the inner expectation is at least its minimum value, which is achieved at $f(x) = P(Y = 1 \mid X = x)$. Since expectation preserves inequalities (if $g(x) \geq h(x)$ for all x, then $\mathbb{E}[g(X)] \geq \mathbb{E}[h(X)]$), the outer expectation is at least the expectation of the pointwise minimum, and choosing $f(x) = P(Y = 1 \mid X = x)$ at every x achieves that bound.

Therefore $f^*(x) = P(Y = 1 \mid X = x)$. For binary Y this determines the whole conditional distribution, since $P(Y = 0 \mid X = x) = 1 - P(Y = 1 \mid X = x)$, so $f^*(x) = P(Y \mid X = x)$ and cross-entropy is a proper scoring rule. ■

1.3 Problem 3

Question. As we saw in class, a requirement for stochastic gradient descent to converge to the minimum of an objective function is that the step size ρ_t decays fast enough that $\sum_t \rho_t^2 < \infty$. In practice it's common to use a constant step size, violating this requirement. Why might this be reasonable in a finite sample setting?

Answer.

The condition $\sum_t \rho_t^2 < \infty$ guarantees that SGD converges to the exact minimizer of the objective as $t \to \infty$. In a finite-sample setting, though, the objective we optimize is the empirical risk, and its minimizer is not the minimizer of the true risk we care about. Because we only have n samples, the empirical minimizer already differs from the true-risk minimizer by a statistical error that doesn't vanish for fixed n (typically on the order of $1/\sqrt{n}$).

With a constant step size ρ, SGD doesn't converge to a single point. Because the gradient noise never shrinks, it settles into a neighborhood of the empirical minimizer whose size depends on ρ. If ρ is chosen so that this neighborhood is small compared to the statistical error, then the extra precision from decaying the step size toward zero barely improves performance on new data. The total error is dominated by the finite sample, not by the optimization.

A constant step size also has practical advantages. Decaying step sizes eventually become very small, so progress slows down a lot, while a constant step reaches the "good enough" region much faster. And since exactly minimizing the empirical risk can lead to overfitting, the leftover noise from a constant step (often combined with early stopping) can act as a form of implicit regularization.

1.4 Coding problems

1.5 Problem 4

Question. One common justification for using square error loss in machine learning is that the residuals of the fitted model should be (approximately) normal. This justification can be approximately tested by fitting a model to the data and then testing the residuals for normality! Fit several ML models to the Average Localization Error (ALE) dataset and run a statistical test to see if the residuals are approximately normally distributed. Predict `ale` using the first 4 variables (ignore `sd_ale`).

Data. The ALE dataset (UCI ML Repository, id 844) is saved in `data/ale.csv`, so the notebook runs without network access. It has 107 rows; the four features are `anchor_ratio`, `trans_range`, `node_density` and `iterations`.

Approach.

- **Models (sklearn):** linear regression, support vector regression (RBF kernel), a decision tree, a random forest and a neural network (MLP). The SVR and MLP use standardized features.
- **Residuals:** $r_i = y_i - \hat{y}_i$, where $\hat{y}_i$ is an *out-of-fold* prediction from 5-fold cross-validation. Each residual comes from a model that did not see observation i, so flexible models can't make their residuals look artificially small or well-behaved by overfitting. In-sample residuals are also reported for comparison.
- **Normality tests:** `scipy.stats.normaltest` (D'Agostino–Pearson K^2, which combines skewness and kurtosis) and the Shapiro–Wilk test, at level $\alpha = 0.05$. The null hypothesis is that the residuals are normally distributed. Q–Q plots show the shape of any departure.

[2]:
```
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_predict
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor

warnings.filterwarnings("ignore", category=UserWarning)

ale = pd.read_csv("data/ale.csv")
features = ["anchor_ratio", "trans_range", "node_density", "iterations"]
X = ale[features].to_numpy()
y = ale["ale"].to_numpy()
print(ale.shape)
ale.head()
```

```
(107, 6)
```

[2]:

	anchor_ratio	trans_range	node_density	iterations	ale	sd_ale
0	30	15	200	40	0.773546	0.250555
1	15	15	100	70	0.911941	0.498329
2	30	15	100	50	0.814867	0.255546
3	15	20	100	20	1.435332	0.394603
4	30	15	100	40	1.265909	0.302943

[3]:
```
models = {
    "Linear regression": LinearRegression(),
    "SVR (RBF)": make_pipeline(StandardScaler(), SVR(C=10, epsilon=0.05)),
    "Decision tree": DecisionTreeRegressor(max_depth=4, random_state=0),
    "Random forest": RandomForestRegressor(n_estimators=300, random_state=0),
    "Neural network (MLP)": make_pipeline(
        StandardScaler(),
        MLPRegressor(hidden_layer_sizes=(32, 32), max_iter=5000,␣
 ↪random_state=0),
    ),
}
cv = KFold(n_splits=5, shuffle=True, random_state=0)


def normality_summary(r):
    return {
        "RMSE": np.sqrt(np.mean(r**2)),
        "skewness": stats.skew(r),
        "excess kurtosis": stats.kurtosis(r),
        "normaltest p": stats.normaltest(r).pvalue,
        "Shapiro-Wilk p": stats.shapiro(r).pvalue,
    }


residuals, rows = {}, []
for name, model in models.items():
    r_oof = y - cross_val_predict(model, X, y, cv=cv)
```

```
    r_in = y - model.fit(X, y).predict(X)
    residuals[name] = r_oof
    rows.append({"model": name, "residuals": "out-of-fold",␣
 ↪**normality_summary(r_oof)})
    rows.append({"model": name, "residuals": "in-sample",␣
 ↪**normality_summary(r_in)})

results = pd.DataFrame(rows).set_index(["model", "residuals"])
results["normal at 5%?"] = np.where(
    (results["normaltest p"] > 0.05) & (results["Shapiro-Wilk p"] > 0.05),␣
 ↪"yes", "no"
)
formats = {"RMSE": "{:.3f}", "skewness": "{:+.2f}", "excess kurtosis": "{:+.
 ↪2f}",
           "normaltest p": "{:.1e}", "Shapiro-Wilk p": "{:.1e}"}
results_table = results.copy()
for col, fmt in formats.items():
    results_table[col] = results_table[col].map(fmt.format)
results_table.columns = ["RMSE", "skew", "ex. kurt", "K2 p", "SW p", "normal?"]
with pd.option_context("display.width", 120):
    print(results_table)
print("\nK2 p = scipy normaltest (D'Agostino-Pearson) p-value; SW p =␣
 ↪Shapiro-Wilk p-value;"
      "\nnormal? = both p-values > 0.05.")
```

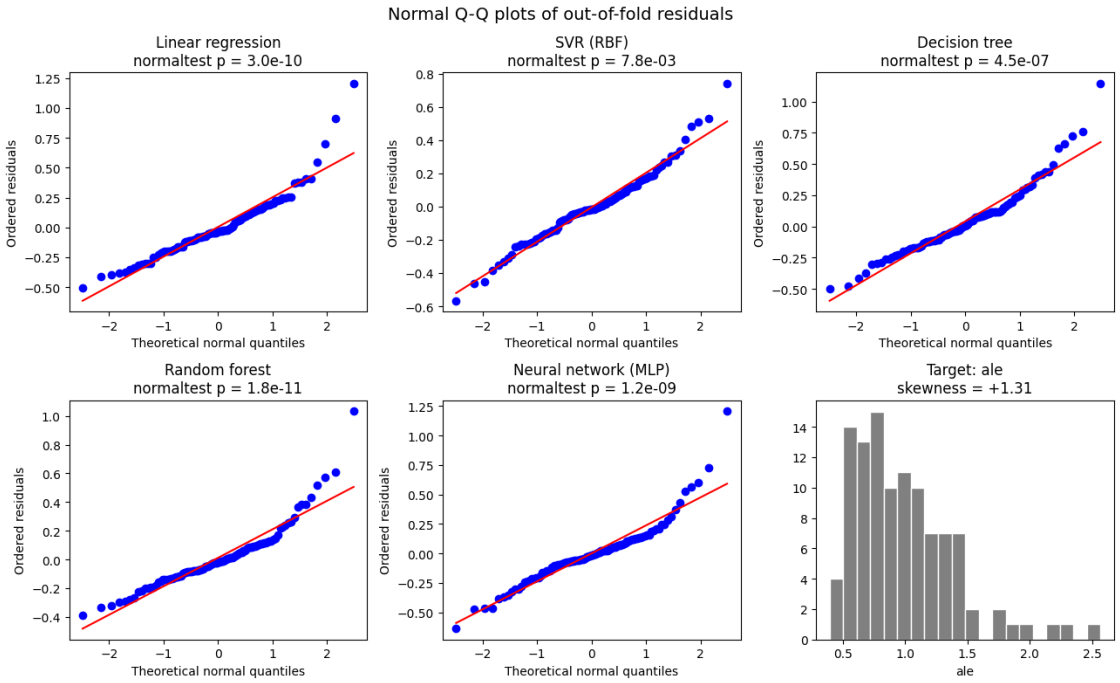
```
                                  RMSE   skew ex. kurt     K2 p     SW p
normal?
model                 residuals
Linear regression     out-of-fold  0.257  +1.46    +4.50  3.0e-10  1.8e-06
no
                      in-sample    0.234  +1.35    +4.40  1.3e-09  5.9e-06
no
SVR (RBF)             out-of-fold  0.207  +0.45    +1.46  7.8e-03  2.5e-02
no
                      in-sample    0.128  +0.41    +5.72  4.2e-06  2.7e-09
no
Decision tree         out-of-fold  0.263  +1.13    +2.67  4.5e-07  6.3e-05
no
                      in-sample    0.144  +0.31    -0.11  3.8e-01  4.6e-01
yes
Random forest         out-of-fold  0.208  +1.62    +5.09  1.8e-11  2.7e-07
no
                      in-sample    0.070  +0.99    +2.72  2.0e-06  3.3e-04
no
Neural network (MLP)  out-of-fold  0.246  +1.26    +5.18  1.2e-09  1.5e-06
no
                      in-sample    0.192  +1.05    +4.69  3.0e-08  5.8e-06
```

no

K2 p = scipy normaltest (D'Agostino-Pearson) p-value; SW p = Shapiro-Wilk p-value;
normal? = both p-values > 0.05.

[4]:
```
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
for ax, (name, r) in zip(axes.flat, residuals.items()):
    stats.probplot(r, dist="norm", plot=ax)
    ax.set_title(f"{name}\nnormaltest p = {stats.normaltest(r).pvalue:.1e}")
    ax.set_xlabel("Theoretical normal quantiles")
    ax.set_ylabel("Ordered residuals")

ax = axes.flat[-1]
ax.hist(y, bins=20, color="gray", edgecolor="white")
ax.set_title(f"Target: ale\nskewness = {stats.skew(y):+.2f}")
ax.set_xlabel("ale")
fig.suptitle("Normal Q-Q plots of out-of-fold residuals", fontsize=14)
fig.tight_layout()
plt.show()
```

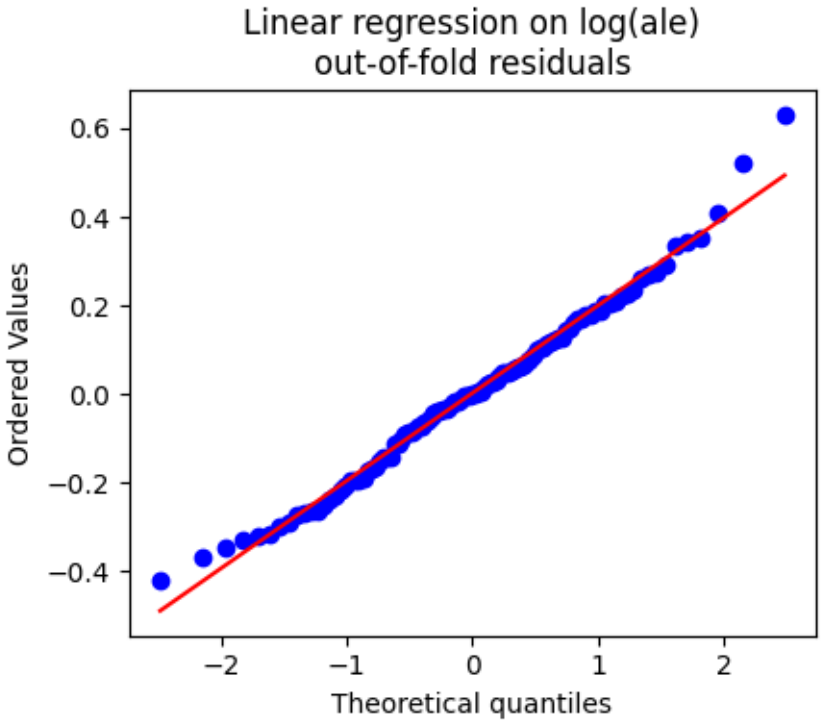


A log-scale check. `ale` is a positive error measure with a long right tail (histogram above), which suggests the noise may be multiplicative rather than additive. As an extra check, fit the same linear model to log(ale) and test those residuals.

[5]:
```
log_y = np.log(y)
r_log = log_y - cross_val_predict(LinearRegression(), X, log_y, cv=cv)
print(pd.Series(normality_summary(r_log)).round(3).to_string())

fig, ax = plt.subplots(figsize=(4.5, 4))
stats.probplot(r_log, dist="norm", plot=ax)
ax.set_title("Linear regression on log(ale)\nout-of-fold residuals")
fig.tight_layout()
plt.show()
```

```
RMSE               0.195
skewness           0.307
excess kurtosis    0.246
normaltest p       0.286
Shapiro-Wilk p     0.502
```



Results and conclusion.

- **Every model rejects normality.** For all five models, both tests reject normality of the out-of-fold residuals at the 5% level. Most p-values are far below 0.05; the SVR comes closest to normal but is still rejected.
- **The departure is the same for every model.** The residuals have positive skewness and positive excess kurtosis. In the Q–Q plots the points follow the line in the middle but bend sharply upward in the right tail: a few observations have much larger `ale` than any model predicts. Changing the model class reduces the RMSE somewhat, but it doesn't remove these

heavy right tails. That suggests the non-normality comes from the noise in the data, not from a poorly chosen model.

- **In-sample residuals can be misleading.** The decision tree's *in-sample* residuals pass both tests, while its out-of-fold residuals clearly fail. When a flexible model fits the training data closely, its in-sample residuals understate the real noise. That is why the out-of-fold residuals are the right ones to test.
- **The log scale tells a different story.** A linear model on log(ale) gives out-of-fold residuals that are consistent with normality (both p-values well above 0.05). The errors look approximately log-normal: roughly normal *multiplicative* noise.

So for predicting `ale` on its original scale, the residuals are not approximately normal: they are skewed and heavy-tailed. This weakens the usual justification for squared error loss, which is sensitive to exactly these large residuals. Two alternatives are a heavier-tailed likelihood, such as the Student-t loss in Problem 5, or modeling log(ale), where squared error is well justified.

1.6 Problem 5

PLACEHOLDER — TO BE ADDED. Problem 5 (automatic differentiation with custom losses) will be added in this section.